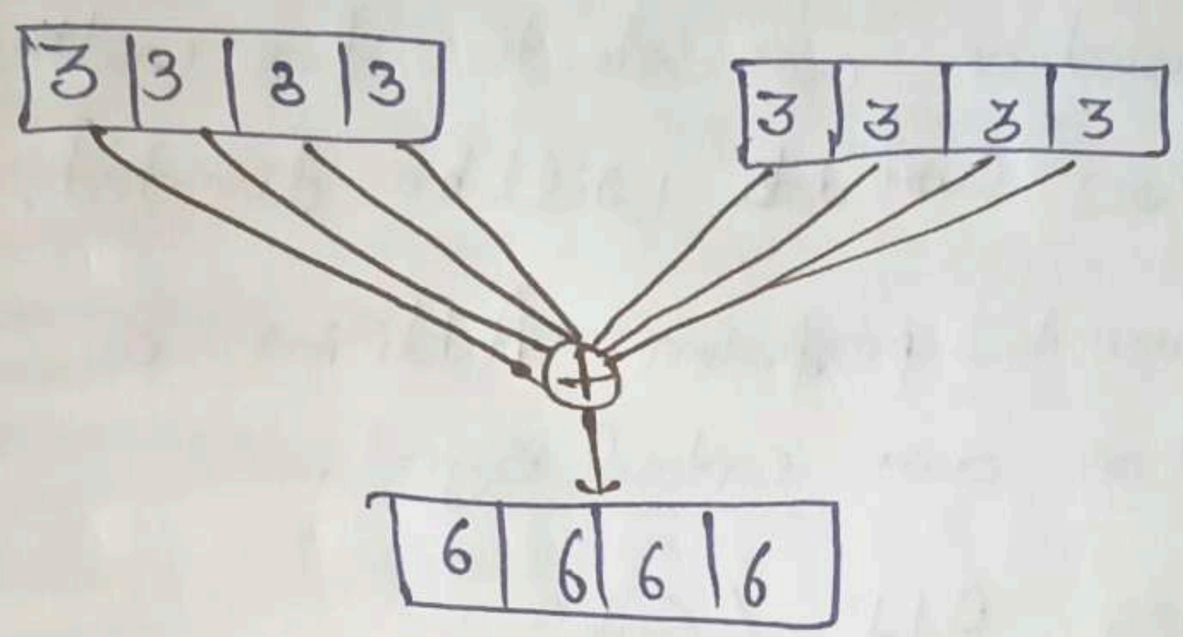
Bonus Question (NPU Assignment)

128 bit Add-Sub SIMD Unit

SIMD - Single Instruction, Multiple data.

It is the form of parallelism, it applies to multiple data.

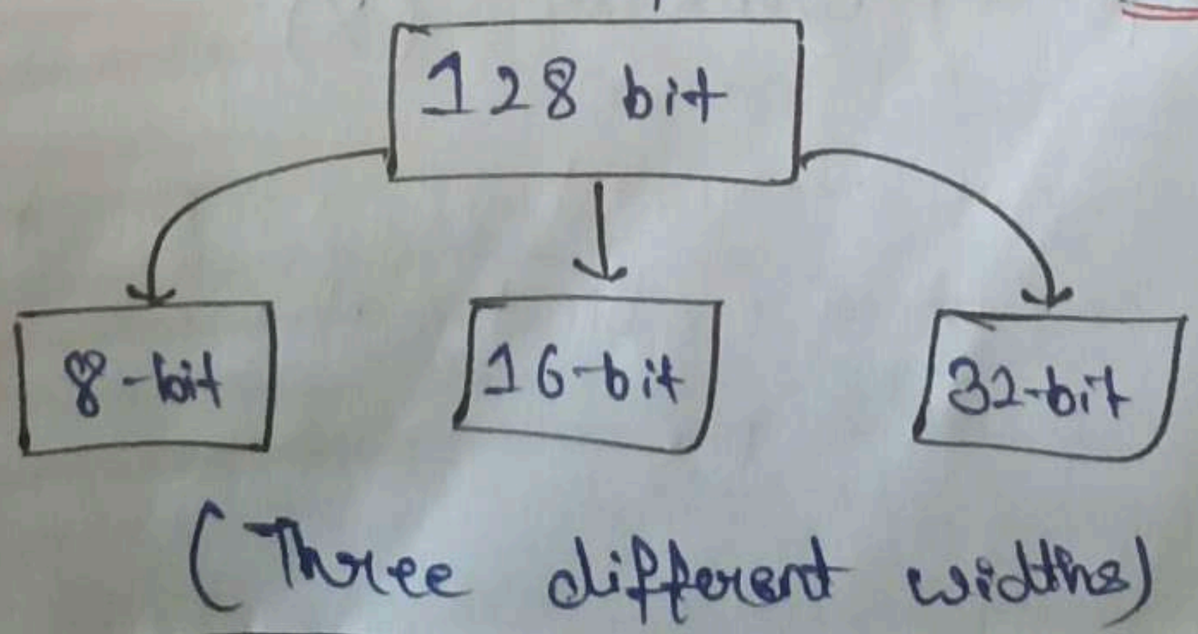
Suppose we want to add two vectors. (group of data)



Here, we have only one instruction per vector of data. The idea is we are applying same operation to all group of data, that's one instruction.

The main aim of this design is to perform addition and subtraction on multiple data values at the same time using a 128 bit SIMD.

Here, instead of performing one operation on the same 128 bit input, we have to divide it into smaller parts called "lanes".



(Three different widths)

For different bit widths, we need different no. of lanes

for 128 bit input.

→ In 8 bit mode, we need $\frac{128}{8} = 16$ lanes.

i.e, 16 operations are to be performed in parallel.

→ In 16 bit mode, we need 8 lanes (128/16)

→ In 32 bit mode, we need 4 lanes.

Therefore, based on our selection for width, we will decide how our data will be divided.

Also we have to perform Addition or Subtraction based on our control signal.

↳ For choosing Add? / Sub?,

we need control signal (one bit)

Variable: sub ⟹ [0 → Addition
1 → Subtraction] Our choice!!

↳ For selecting width, we will be needing one more control signal (width_selection), to select three widths, for that we require two bit signal.

[00: 8-bit mode (16 lanes)
01: 16-bit mode (8 lanes)
10: 32-bit mode (4 lanes)
11: Don't care (X)]

↳ This is our design choice, we can choose any other combination, it won't affect our concept.

* The modes also relate to each other,

The low 32 bits [31:0] are one lane in 32-bit mode, two lanes ([15:0] and [31:16]) in 16-bit mode, and four lanes in 8 bit mode.

The core idea

- The unit shall support Saturating arithmetic

These are the saturation limits:

8-bit mode: min = 0, max = 255 (8'hFF)
16-bit mode: min = 0, max = 65535 (16'hFFFF)
32-bit mode: min = 0, max = 4294967295 (32'hFFFFFFFF)

Ex: 250+20 (8 bit addition)
output = 255 (saturated)

- A and B never physically change, the order in which we read them usually is changing here: 16 small no; 8 medium numbers or 4 large numbers. Bits are same, only our grouping here is different.
- Basic rule:- Lane 0 will always sit at the least significant end, and lane numbers increase towards the MSB,

Suppose 'i' is the lane number.

8-bit mode: Lane 'i' occupies [8i+7 : 8i]
16-bit mode: Lane 'i' occupies [16i+15 : 16i]
32-bit mode: Lane 'i' occupies [32i+31 : 32i]

Example: For lane 2, i = 2
8-bit mode, [23:16]
16-bit mode, [16×2+15 : 16×2] = [47:32]

Selection of mode

All the modes may have different answers based on their i/p.

For example; FF + 01 ⇒ Saturates for 8-bit mode
but it doesn't saturate for 16-bit / 32-bit mode

↳ So, we will be computing all three groupings in parallel and based on our mode selected (width-selection) the control signal then choose which grouping's result will be sending to the ✱ output.

Our design choice

The division of lane will happen in the structure of hardware, and the control signal only picks among the three sets of lane result.

we can understand this with the help of this example,

Example

If we set upper 96 bits of both operands to '0,

A[31:0] = 0x00FFFFFF
B[31:0] = 0x00010001

Sub = 0 (addition)

1) In 8-bit mode

We have 4 relevent lanes.

Lane 3 A[31:24] = 0x00,
B[31:24] = 0x00.
Sum = 0x000 ⇒ Result = 0x00 (no overflow)

Lane 2 $A[23:16] = 0xFF$, $B[23:16] = 0x01$

True sum = 0x100

(carry=1) but H will saturate,

Result = 0xFF (Saturates to 255)

Lane 1 $A[15:8] = \sout{0x00}\ 0xFF$, $B[15:8] = 0x00$

Result = 0xFF (fits)

Lane 0 $A[7:0] = 0xFF$, $B[7:0] = 0x01$

Result = 0xFF (Saturates to 255)

∴ output, $Y[31:0] = \sout{0xFF}$

$Y[31:0] = 0x00FFFFFF$

2) For ~~32-bit~~ 16 bit mode (2 relevent lanes)

Lane 1 $A[31:16] = 0x00FF$, $B[31:16] = 0x0001$

True Sum = 0x0100 (No saturation)

Lane 0 $A[15:0] = 0xFFFF$, $B[15:0] = 0001$

Sum = 0x1-0000 (saturates)

Result = 0xFFFF (65535)

Output, $Y[31:0] = 0x0100\text{-}FFFF$

③ For 32-bit mode

only one relevant lane

$A[31:0] = 0x00FFFFFF$, $B[31:0] = 0x0001\text{-}0001$

Result = 0x0101-0100 (No saturation)

$Y[31:0] = 0x0101_0000$

Clearly from example,

our output in all three modes are different.

∴ output depends on what type of grouping mode we will select.

That's why, we have to make a design that computes all three results at once from same A and B and 'Sub' (0 or 1). The width-selection signal will only decide, what output will be.

i.e, which result will reach y.

Therefore, "The division will happen in structure."

Verilog logic for Subtraction

For 8-bit mode

```
for (i=0; i<16; i=i+1) begin
if (Sub==1'b0) begin
   sum8 = {1'b0, A[i*8+7]} + {1'b0, B[i*8+:8]};
```

It extends the operand to 9 bit just in case an extra bit occurs.

```
   if (sum_8[8])  ⟶ If 9th bit is one
      Y[i*8+:8] = 8'hFF; } o/p result sets to 255
   end
else begin
      if (A[i*8+:8] < B[i*8+:8])
         Y[i*8+:8] = 8'h00 // negative sets to zero
      else
         Y[i*8+:8] = A[i*8+:8] - B[i*8+:8];
   end
end
end
```

Subtraction